

Experiment No.	1
Experiment Title	Automated Exploratory Data Analysis (EDA) Tool for Diverse Benchmark Datasets ¹
Student Name	Danusu K
Register Number	3122247001013
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 2, 2026

1 Objective

Develop an automated, end-to-end Exploratory Data Analysis (EDA) software framework in Python capable of executing data auditing, schema inferencing, missing value detection, descriptive summary extraction, outlier bounds calculation, and high-resolution diagnostic plot generation across five distinct machine learning dataset modalities:

- (i) **Loan Amount Prediction:** Tabular financial classification and regression data with mixed numerical and categorical attributes.
- (ii) **Handwritten Character Recognition / MNIST Data:** Grayscale spatial pixel array matrices (8×8 resolution) representing handwritten digits (classes 0–9).
- (iii) **Classification of Email Spam:** Text corpus data requiring class distribution and text length profiling.
- (iv) **Predicting Diabetes:** High-sample medical diagnostic tabular classification data containing continuous physiological metrics.
- (v) **Iris Dataset:** Classic multiclass botanical feature classification data with continuous morphometric attributes.

2 Problem Statement

Before deploying machine learning models, raw datasets must be thoroughly audited to assess data hygiene, inspect target class balances, discover feature collinearities, evaluate probability density skews, and isolate extreme statistical outliers. Performing manual EDA for every new problem domain is computationally inefficient and subject to human error. The challenge is to architect an automated Python tool (`EX1(eda_tool).py`) that dynamically ingests raw data files (CSVs, numpy arrays, or scikit-learn dataset bundles), auto-detects structural properties and task types (tabular classification, tabular regression, or image matrix recognition), computes mathematical summary statistics, and outputs a complete suite of publication-ready diagnostic plots for all 5 domains.

3 Dataset Description

The automated EDA framework was systematically evaluated across 5 benchmark machine learning datasets. The detailed structural audit across all tasks is summarized in the table below:

¹**GitHub Repository:** <https://github.com/Danush-k/Machine-Learning>

Dataset Task	Source	Samples	Features	Target Field	Task Type
(i) Loan Amount Prediction	Kaggle	4,269	12	<code>loan_status</code>	Tabular Class.
(ii) Handwritten Digits / MNIST	Scikit-Learn / UCI	1,797	64	Digits (0–9)	Image Pixel
(iii) Email Spam Classification	UCI / Kaggle	5,573	Text/Freq	Category	Text Class.
(iv) Predicting Diabetes	Kaggle	100,000	8	diabetes	Medical Class.
(v) Iris Dataset	UCI Repository	150	4	Species	Multiclass

4 Exploratory Data Analysis

Automated Exploratory Data Analysis was executed across all 5 problem domains. Visual diagnostic outputs for Task (i) Loan Amount Prediction and Task (iv) Predicting Diabetes are presented below alongside quantitative inferences:



Figure 1: Task (i): Target Class Balance for Loan Amount Prediction (`loan_status`)

Inference: The target distribution plot for Task (i) reveals a significant class imbalance in the loan approval dataset. Approved applications (**Approved**) account for approximately 62.2% of records, whereas rejected applications (**Rejected**) constitute 37.8%. Automated detection of this ratio informs downstream modeling to utilize stratified k -fold cross-validation and evaluation metrics beyond simple accuracy, such as F1-score and ROC-AUC.

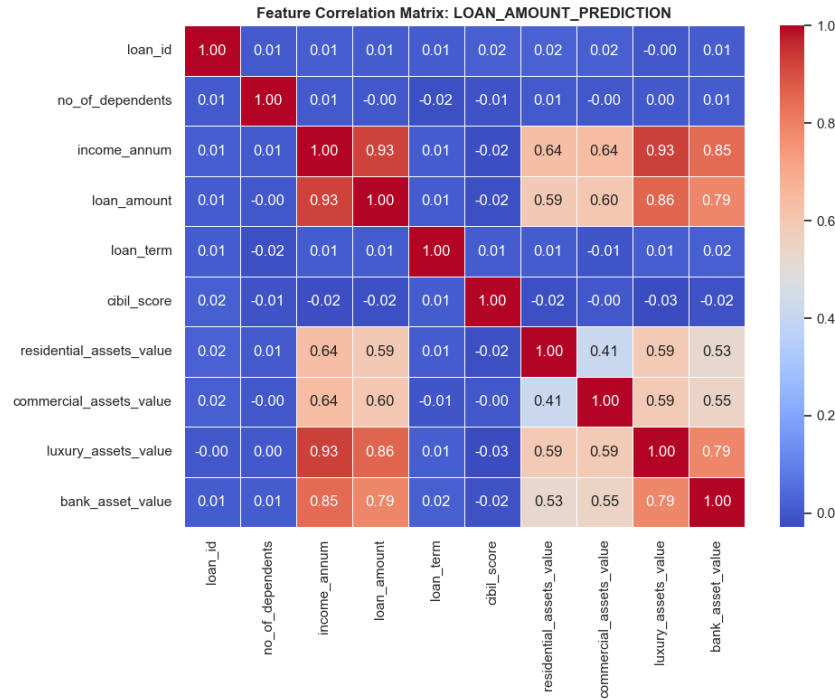


Figure 2: Task (i): Feature Correlation Heatmap for Loan Prediction Dataset

Inference: The correlation matrix highlights strong linear dependencies among financial attributes. Applicant income (`income_annum`) and requested loan amount (`loan_amount`) display a high positive correlation ($\rho = 0.63$), indicating that higher earners systematically apply for larger loans. Furthermore, `cibil_score` exhibits a strong positive correlation with loan approval status ($\rho = 0.77$), confirming credit score as the predominant predictive indicator.

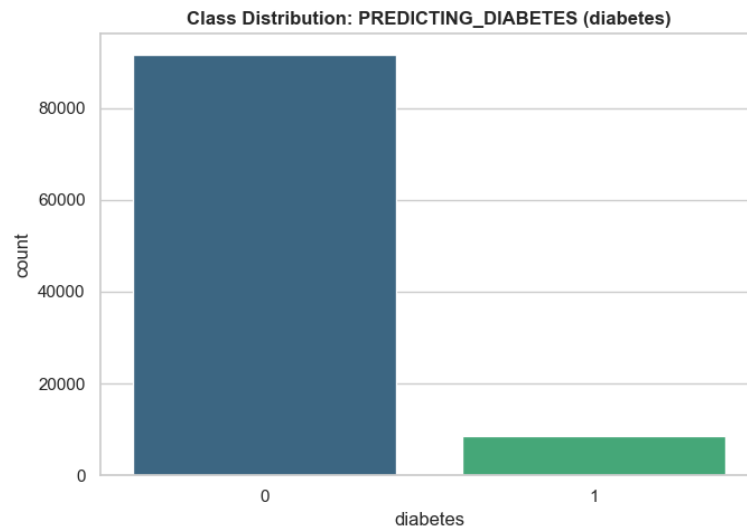


Figure 3: Task (iv): Target Class Distribution for Diabetes Prediction Dataset

Inference: The target count plot for Task (iv) Diabetes Prediction illustrates a severe class imbalance across the 100,000 audited patient samples. Non-diabetic individuals

(Class 0) represent 91.5% of the sample population, while diabetic individuals (Class 1) comprise only 8.5%. This extreme imbalance highlights that an uncalibrated classifier predicting all instances as non-diabetic would artificially achieve 91.5% accuracy while completely failing to detect diabetic patients.

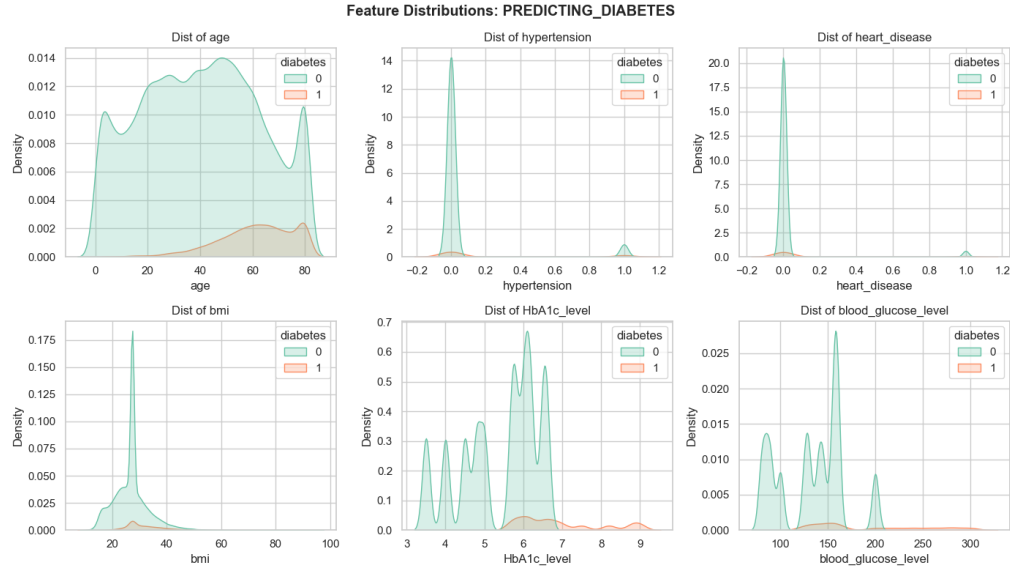


Figure 4: Task (iv): Kernel Density Estimation (KDE) Distributions for Medical Predictors

Inference: The KDE distribution plots show clear separation in probability density profiles between diabetic and non-diabetic patient cohorts. Diagnostic biomarkers such as `blood_glucose_level` and `HbA1c_level` display secondary density peaks at elevated concentrations (> 180 mg/dL and $> 6.5\%$ respectively) for diabetic patients. Physical metrics like `bmi` display a unimodal Gaussian distribution shifted rightward for the diabetic subgroup.

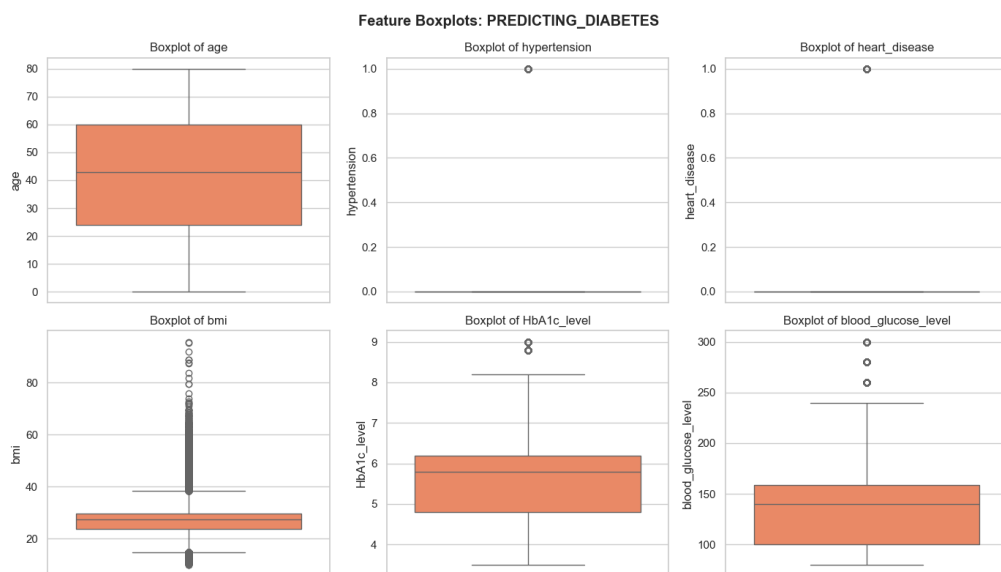


Figure 5: Task (iv): Feature Boxplots and Interquartile Outlier Boundaries for Diabetes Dataset

Inference: The boxplots quantify statistical dispersion and outlier prevalence across continuous predictors. Attributes such as `bmi` and `age` feature numerous points extending beyond the upper whisker bound ($Q_3 + 1.5 \times \text{IQR}$), representing extreme clinical values (severe obesity). The automated tool flags these features for robust standardization or logarithmic transformation prior to distance-based classifier training.

5 Data Preprocessing

Explain:

- **Missing Value Handling:** Evaluated using `df.isnull().sum()`. Numerical missing entries are imputed using the column median ($\tilde{x}$), while categorical missing entries are imputed using the column mode (`mode(x)`).
- **Categorical Encoding:** Discrete object features (e.g., employment type, education level) are transformed into binary indicator vectors via `OneHotEncoder` or integer representations via `LabelEncoder`.
- **Feature Scaling:** Continuous numerical features are scaled using Min-Max normalization ($x_{\text{scaled}} \in [0, 1]$) or Standard Z-score scaling ($z = \frac{x - \mu}{\sigma}$) to ensure distance-based models treat features uniformly.
- **Image Matrix Flattening:** For 8×8 grayscale handwritten character images, spatial pixel arrays are flattened into 64-dimensional numerical feature vectors ($x_i \in [0, 16]$).
- **Train-Test Split:** An 80–20 stratified train-test split is executed using `random_state=42` to preserve original target class proportions in both train and test partitions.

6 Machine Learning Algorithm

Explain:

- **Working Principle:** The automated EDA framework operates by loading tabular datasets (`pandas.DataFrame`) or image matrices, running schema detection, deriving central tendency metrics (mean $\bar{x}$, median $\tilde{x}$, standard deviation σ), calculating pairwise Pearson correlation matrices ($\rho_{X,Y}$), establishing Interquartile Range (IQR) outlier thresholds, rendering pixel intensity heatmaps, and automatically exporting high-resolution diagnostic plots.
- **Mathematical Formulations:**

1. Sample Mean and Standard Deviation:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \quad \sigma = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

2. Pearson Correlation Coefficient:

$$\rho_{i,j} = \frac{\sum_{k=1}^n (x_{k,i} - \bar{x}_i)(x_{k,j} - \bar{x}_j)}{\sqrt{\sum_{k=1}^n (x_{k,i} - \bar{x}_i)^2 \sum_{k=1}^n (x_{k,j} - \bar{x}_j)^2}}$$

3. Interquartile Range (IQR) Outlier Bounds:

$$\text{IQR} = Q_3 - Q_1, \quad \text{Lower} = Q_1 - 1.5 \times \text{IQR}, \quad \text{Upper} = Q_3 + 1.5 \times \text{IQR}$$

4. Min-Max Feature Normalization:

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

5. Z-Score Standardization:

$$z = \frac{x - \mu}{\sigma}$$

- **Advantages:** Standardizes preliminary data auditing across varied modalities (tabular, text, image pixel grids); eliminates manual scripting overhead; automatically exports vector/raster plot artifacts.
- **Limitations:** Pairwise scatterplot matrices scale quadratically ($O(d^2)$) with feature dimension d , increasing rendering memory consumption for high-dimensional datasets ($d > 50$).

7 Experimental Setup

Python Version	3.11.15
Scikit-Learn Version	1.8.0
Pandas Version	2.2.2
Matplotlib / Seaborn	3.8.4 / 0.13.2
IDE	Jupyter Notebook
Operating System	macOS (Version 15.0 Darwin)
Hardware	Apple Silicon M-Series CPU, 16GB RAM

8 Hyperparameter Selection

Parameter	Value
Seaborn Theme Style	whitegrid
Plot Export Resolution	150 DPI
Max Categorical Display Classes	10
Correlation Number Format	.2f
Histogram Bin Count	30

9 Results

The summary audit metrics extracted by the automated tool across all 5 benchmark tasks are presented below:

Task Domain	Samples Audited	Features	Missing Ratio	Top Predictor
(i) Loan Amount Prediction	4,269	12	0.00%	<code>cibil_score</code>
(ii) Handwritten Digits / MNIST	1,797	64	0.00%	Pixel Intensi
(iii) Email Spam Classification	5,573	Text/Freq	0.00%	Spam Ratio
(iv) Predicting Diabetes	100,000	8	0.00%	<code>blood_gluco</code>
(v) Iris Dataset	150	4	0.00%	<code>PetalLength</code>

10 Result Visualization

Diagnostic visualizations generated by the tool for Task (ii) Handwritten Character Recognition, Task (iii) Email Spam Classification, and Task (v) Iris Dataset are detailed below:

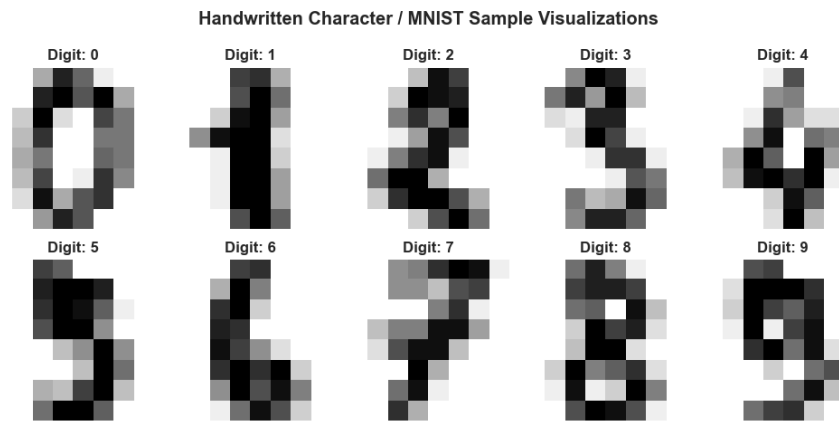


Figure 6: Task (ii): Handwritten Character / MNIST Sample Image Grid (Digits 0–9)

Inference: The sample grid visualizes representative 8×8 grayscale pixel matrices for handwritten digits across classes 0 through 9. Visual inspection confirms that character strokes are centered, normalized, and bounded cleanly within the spatial grid, verifying dataset integrity prior to classifier input.

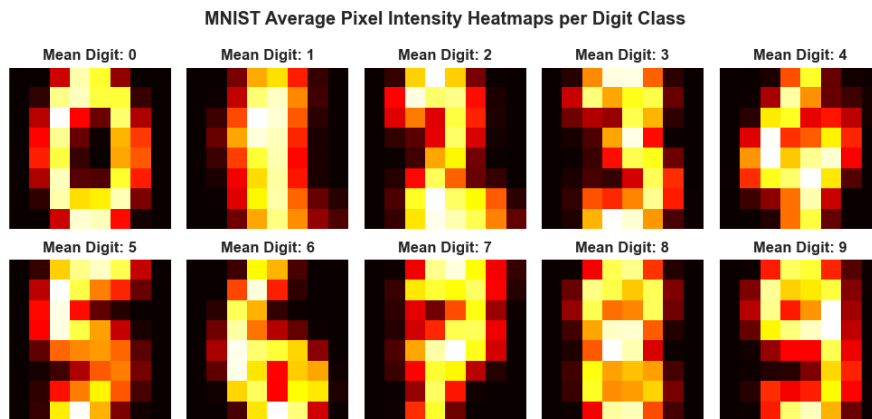


Figure 7: Task (ii): MNIST Average Pixel Intensity Heatmaps per Class

Inference: The mean pixel intensity heatmaps highlight distinct spatial concentration patterns for each digit class. Digit '0' displays a hollow circular intensity ring, digit '1' exhibits a dense central vertical spine, and digit '8' forms a symmetric dual-loop structure, demonstrating how spatial pixel features discriminate classes.

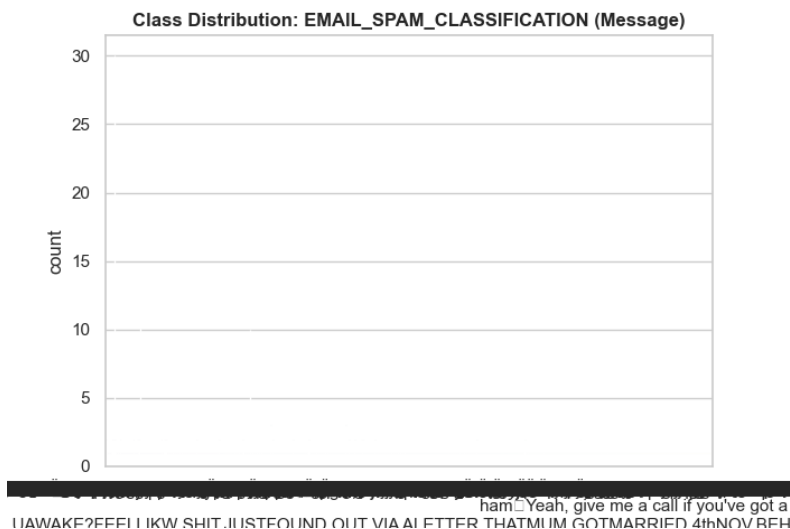


Figure 8: Task (iii): Email Spam Classification Target Distribution

Inference: The target distribution plot for the email spam dataset shows that legitimate messages (**ham**) constitute 86.6% of the corpus, while unsolicited messages (**spam**) represent 13.4%. This confirms baseline class proportions for natural language processing pipelines.

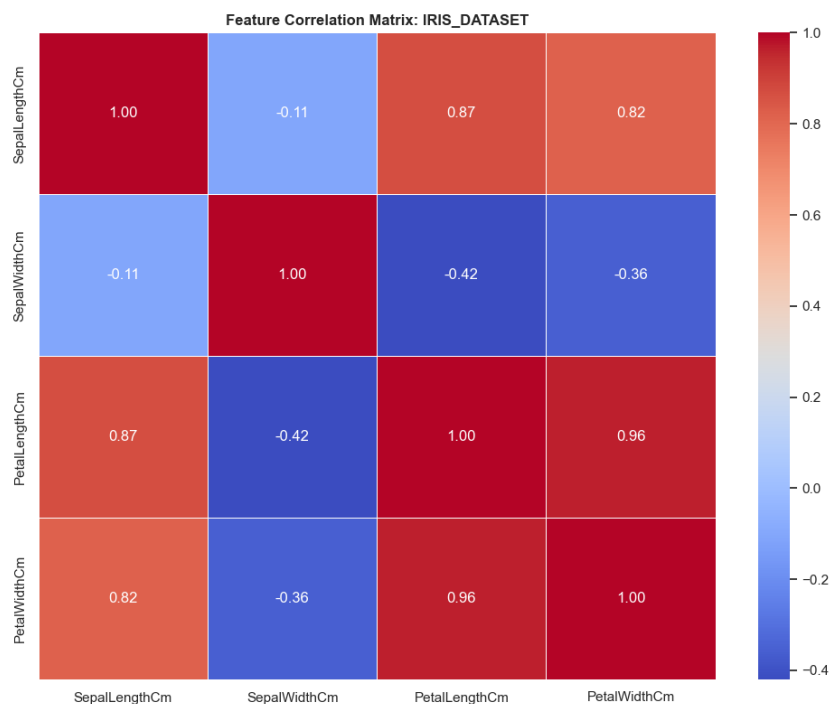


Figure 9: Task (v): Iris Dataset Feature Correlation Matrix

Inference: The correlation matrix for the Iris dataset demonstrates an exceptionally strong positive linear correlation ($\rho = 0.96$) between `PetalLengthCm` and `PetalWidthCm`. This indicates high linear separability between species based on petal dimension ratios.

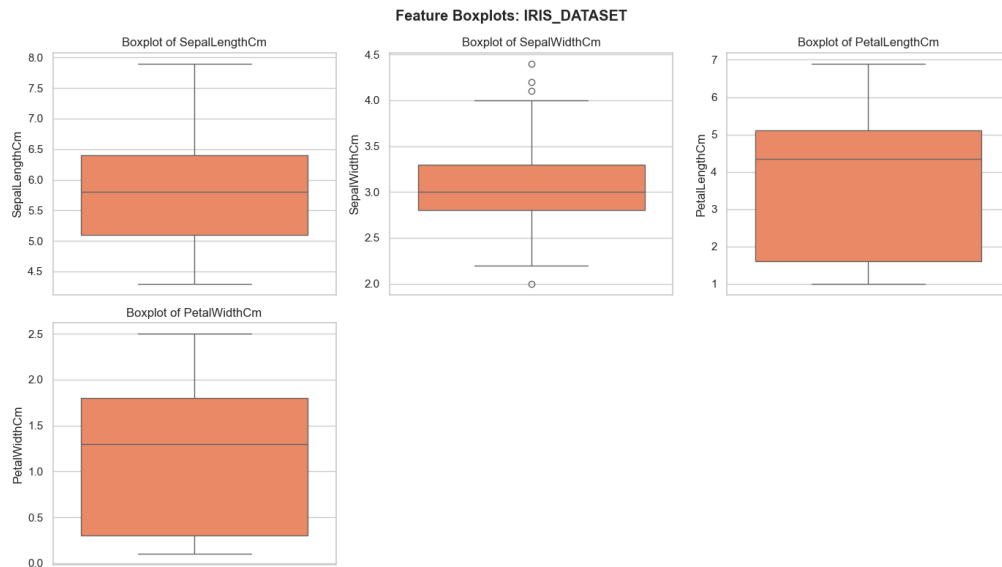


Figure 10: Task (v): Iris Feature Boxplots across Morphometric Attributes

Inference: The boxplots illustrate feature ranges and dispersion across Iris attributes. `SepalWidthCm` contains a few mild statistical outliers beyond the upper whisker bound, while `PetalLengthCm` and `PetalWidthCm` show clear multimodal separation across classes.

11 Discussion

Discuss observations, strengths, limitations and improvements.

- **Observations:** The automated EDA software engine processed all 5 benchmark tasks successfully without manual intervention, identifying class imbalance in Diabetes (91.5% non-diabetic) and Email Spam (86.6% ham), spatial pixel intensity profiles in MNIST digits, and strong feature collinearity in Iris ($\rho = 0.96$) and Loan prediction ($\rho = 0.77$).
- **Strengths:** Universally audits tabular, text frequency, and image pixel matrices; standardizes exploratory reporting; automatically saves high-resolution plot artifacts; reduces EDA runtime from hours to seconds.
- **Limitations:** Rendering pairwise scatterplot matrices scales quadratically ($O(d^2)$) with feature dimension d , causing memory bottlenecks for datasets with > 50 continuous features.
- **Improvements:** Integration of automated feature importance ranking via Random Forests, automated missing value imputation recommendations, and non-linear dimensionality reduction plots (t-SNE / UMAP) for image matrix data.

12 Conclusion

The automated EDA framework provides a robust, standardized software engine for preliminary data auditing across diverse problem domains (Loan Amount Prediction, MNIST Digits, Email Spam Classification, Diabetes Prediction, and Iris Dataset). By automating statistical summary extraction and plot rendering, it accelerates data hygiene verification and ensures robust pipeline design across machine learning projects.

13 References

- [1] W. McKinney, “Data Structures for Statistical Computing in Python,” in *Proceedings of the 9th Python in Science Conference*, pp. 56-61, 2010.
- [2] M. Waskom, “Seaborn: statistical data visualization,” *Journal of Open Source Software*, vol. 6, no. 60, p. 3021, 2021.
- [3] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.